

Predicting Food Delivery Time with Machine Learning

A Regression Study on Zomato Delivery Operations Data

Objective: Build and deploy a regression model that estimates food delivery time (in minutes) for online food delivery platforms such as Zomato and Swiggy.

Final result: Test $R^2 = 0.8302$, RMSE = **3.864 min**, MAE = **3.11 min** — exceeding the published LightGBM benchmark of $R^2 = 0.76$ on the same dataset.

Table of Contents

- 1. Problem Statement
- 2. Dataset Selection
- 3. Data Quality and Cleaning
- 4. Exploratory Data Analysis
- 5. Feature Engineering
- 6. Feature Selection
- 7. Encoding and Data Splitting
- 8. Model Training
- 9. Model Selection
- 0. Final Evaluation
 - 1. Benchmark Comparison
 - 2. Deployment Plan
 - 3. Limitations and Future Work
 - 4. Conclusion

1. Problem Statement

Online food delivery platforms must display accurate delivery-time estimates to maintain customer trust and manage expectations. The estimate must account for factors outside the platform’s control — traffic congestion, weather, festival demand surges, and the physical distance between restaurant and customer.

This project frames delivery-time estimation as a **supervised regression problem**:

- **Target variable:** `Time_taken (min)` — a continuous value representing actual delivery duration
- **Predictors:** delivery-person attributes, geospatial coordinates, temporal features, and environmental conditions
- **Evaluation metrics:** R^2 (proportion of variance explained), RMSE (root mean squared error, penalises large errors), and MAE (mean absolute error, interpretable directly in minutes)
- **Deliverable:** a trained model plus a deployable prediction service

2. Dataset Selection

Candidates Considered

Two Kaggle datasets were evaluated:

Dataset	Structure	Verdict
<code>saurabhbadole/zomato-delivery-operations-analytics-dataset</code>	Single fully-labelled CSV	Selected
<code>gauravmalik26/food-delivery-dataset</code>	Split into <code>train.csv</code> / <code>test.csv</code>	Rejected

Rationale

Both datasets contain essentially the same underlying Zomato delivery records with identical columns. The decisive difference is packaging.

The second dataset's `test.csv` **omits the target column** `Time_taken (min)` — it is structured for Kaggle-competition-style submissions where ground truth is deliberately withheld. For a build-and-deploy project requiring the ability to measure error on held-out data, that file is unusable. Only its `train.csv` would have been viable, which reduces it to a strictly worse version of the first option.

The first dataset provides a single file where every row carries the target, allowing full control over train/validation/test splitting.

Dataset Description

- **Rows:** 45,584
- **Columns:** 20
- **Domain:** food deliveries across multiple Indian cities

Column	Description	Type
ID	Unique delivery identifier	Identifier
Delivery_person_ID	Unique delivery-person identifier	Identifier
Delivery_person_Age	Age of the delivery person	Numeric
Delivery_person_Ratings	Platform rating of the delivery person	Numeric
Restaurant_latitude	Restaurant latitude	Geospatial
Restaurant_longitude	Restaurant longitude	Geospatial
Delivery_location_latitude	Delivery latitude	Geospatial
Delivery_location_longitude	Delivery longitude	Geospatial
Order_Date	Date the order was placed	Temporal
Time_Orderd	Time the order was placed	Temporal
Time_Order_picked	Time the order was collected	Temporal
Weather_conditions	Weather at delivery time	Categorical
Road_traffic_density	Traffic congestion level	Categorical
Vehicle_condition	Condition rating of the vehicle (0/1/2)	Categorical
Type_of_order	Snack / Meal / Drinks / Buffet	Categorical
Type_of_vehicle	Motorcycle / scooter / electric scooter	Categorical
multiple_deliveries	Orders bundled into one trip (0-3)	Numeric (count)
Festival	Whether delivery fell on a festival	Binary
City	Metropolitan / Urban / Semi-Urban	Categorical
Time_taken (min)	Target — actual delivery duration	Numeric

3. Data Quality and Cleaning

Missing Values

Eight columns contained nulls:

Column	Nulls	% of data	Strategy	Rationale
Delivery_person_Ratings	1,908	4.19%	Median	Robust to the low-rating tail
Delivery_person_Age	1,854	4.07%	Median	Robust to skew
Time_Orderd	1,731	3.80%	Drop rows	Timestamps cannot be meaningfully imputed
City	1,200	2.63%	Mode	Small proportion, categorical
multiple_deliveries	993	2.18%	Fill with 0	Missing plausibly means “no bundling”

Weather_conditions	616	1.35%	Mode	Small proportion, categorical
Road_traffic_density	601	1.32%	Mode	Small proportion, categorical
Festival	228	0.50%	Mode	Small proportion, binary

Note the deliberate choice for `multiple_deliveries`: rather than statistically guessing with a median or mode, missingness was treated as **semantically meaningful** — an absent bundling count most plausibly indicates no additional deliveries. Filling with `0` preserves that meaning.

An initial attempt to impute `Time_Orderd` by subtracting an average preparation gap from `Time_Order_picked` proved fragile, because `Time_Order_picked` itself failed to parse for many rows (see §5.2). The 1,731 affected rows were dropped instead — a defensible trade at under 4% of the data.

Rows after cleaning: 43,853

Corrupted Geospatial Coordinates

Inspection of the coordinate ranges revealed two distinct defects:

Defect	Rows affected	Fix
Sign-flipped latitudes (e.g. <code>-30.90</code> instead of <code>30.90</code>)	156	Applied <code>abs()</code> — magnitudes were correct
Near-zero placeholders (<code>0.0</code> , <code>0.01</code>)	3,509	Dropped — distance is uncomputable

India's valid coordinate envelope is approximately latitude 8–37 and longitude 68–97. Values outside this range are unrecoverable rather than merely noisy, so rows containing them were removed.

Rows after coordinate cleaning: 40,344

4. Exploratory Data Analysis

4.1 Target Variable

Summary statistics for `Time_taken (min)` on 43,853 rows:

Statistic	Value
Mean	26.29
Median	26.00
Std. dev.	9.37
Min	10
Max	54
25th percentile	19
75th percentile	32
Skewness	0.483
Kurtosis	−0.314

Three conclusions followed:

No transformation required. Skewness of 0.483 falls within the conventional $|\text{skew}| < 0.5$ threshold for approximate symmetry, and negative kurtosis indicates *lighter* tails than a normal distribution. A log transform of the target was therefore unnecessary.

Apparent outliers are legitimate. Boxplot whiskers flagged a handful of points near 52–54 minutes, but with a range of 10–54 minutes these represent genuinely slow deliveries — plausibly festival plus traffic jam plus long distance. Removing them would discard precisely the hard cases the model must learn. They were retained. Zero non-positive values were present.

The distribution is bimodal. The histogram showed distinct peaks near 18 and 26 minutes with a visible trough between. This was not noise. Overlaying `Road_traffic_density` on the distribution confirmed the mechanism: **Low-traffic deliveries** peak sharply around 15–20 minutes, while **Jam-traffic deliveries** shift right to peak around 25–30 minutes with a tail beyond 50. The two

clusters, superimposed, produce the bimodal shape.

This finding carried a direct modelling implication, recorded early and later confirmed: **a linear model cannot represent two distinct regimes with a single fitted line**. Tree-based methods, which split on categorical thresholds, should be expected to outperform substantially.

4.2 Feature Distributions

Feature	Distribution	Observation
Delivery_person_Age	Uniform, 20–39	Suspiciously flat — no natural peak
Delivery_person_Ratings	Left-skewed, clustered 4.5–5.0	Thin tail down to 2.5
Weather_conditions	Near-uniform (7,107–7,475 per class)	Artificially balanced
Type_of_order	Near-uniform (10,853–11,088)	Artificially balanced
Vehicle_condition	Near-uniform (14,599–14,639)	Artificially balanced
Road_traffic_density	Low 15,061 / Jam 13,796 / Medium 10,676 / High 4,320	Realistic skew
Type_of_vehicle	motorcycle 25,627 / scooter 14,692 / electric_scooter 3,534	Realistic skew
multiple_deliveries	1 → 27,060 / 0 → 14,530 / 2 → 1,920 / 3 → 343	Realistic decay
Festival	No 42,996 / Yes 857	2% positive class
City	Metropolitan 33,940 / Urban 9,757 / Semi-Urban 156	Severe imbalance

Several columns exhibit near-perfect class balance, a fingerprint of synthetic data generation rather than organic collection. This observation was noted but — importantly — **did not by itself justify dropping any feature** (see §4.3).

`City` presented a real structural risk: with only 156 Semi-Urban rows (0.36%), a naive random split could leave the category badly under-represented in one partition. This motivated stratified splitting (§7.2).

4.3 Feature-Target Relationships

Group means of `Time_taken (min)` by category:

Road_traffic_density

Category	Mean	Count
Jam	31.15	13,796
High	27.21	4,320
Medium	26.72	10,676
Low	21.28	15,061

Spread: 9.9 min. Note that Medium and High are nearly tied, so the ordering is not a clean monotonic staircase.

Festival

Category	Mean	Count
Yes	45.50	857
No	25.91	42,996

Spread: 19.6 min — an enormous effect confined to 2% of rows.

City

Category	Mean	Count
Semi-Urban	49.74	156
Metropolitan	27.13	33,940
Urban	23.00	9,757

Spread: 26.7 min — the largest single effect in the dataset, but resting on only 156 observations for the extreme category.

multiple_deliveries

Count	Mean
0	22.85
1	26.86
2	40.43
3	47.82

A clean, monotonic 25-minute staircase — the cleanest ordinal relationship in the data.

Weather_conditions

Category	Mean
Cloudy	28.94
Fog	28.93
Windy	26.10
Sandstorms	25.89
Stormy	25.86
Sunny	21.85

Spread: 7.1 min. Balance in the *feature* did not preclude a real effect on the *target*.

Vehicle_condition — a critical finding

Condition	Mean
0	30.05
1	24.37
2	24.47

The relationship is **non-monotonic**: it drops sharply from 0 to 1, then rises marginally. Because the column was stored as `int64`, a linear model would fit a straight line through a genuinely non-linear pattern. This directly determined the encoding decision (§7.1): **one-hot, not ordinal**.

Type_of_order — no signal

Category	Mean
Meal	26.42
Buffet	26.29
Snack	26.27
Drinks	26.20

A spread of **0.22 minutes** across 43,853 rows. Statistically indistinguishable from zero. Dropped.

Delivery_person_Ratings — non-linear

Binned means reveal a decline followed by a plateau:

Rating bin	Mean time	Count
(2.5, 3.5]	36.83	452
(3.5, 4.0]	35.88	1,934
(4.0, 4.5]	30.56	8,919
(4.5, 4.8]	24.28	21,492
(4.8, 5.0]	24.63	11,036

Time falls steeply until a rating of 4.5, then **flattens and ticks marginally upward**. Above 4.5 — which covers 74% of the data — ratings barely discriminate. A straight regression line through this curve materially misrepresents the relationship.

Delivery_person_Age — a corrected expectation

Age was initially expected to carry little signal, on the grounds that its uniform distribution suggested synthetic generation. **This expectation was wrong.** The regression fit rose from roughly 21 minutes at age 20 to roughly 31 minutes at age 39 — a 10-minute swing, later corroborated by a Pearson correlation of 0.302 and a top-five feature-importance ranking.

Methodological lesson: the marginal distribution of a feature says nothing about its relationship to the target. Uniformity in x does not imply independence from y . This distinction is recorded because the initial reasoning conflated the two.

5. Feature Engineering

5.1 Geospatial: Haversine Distance

The dataset supplies raw coordinates but no distance measure. Great-circle distance between restaurant and delivery point was computed using the haversine formula:

```
def haversine(lat1, lon1, lat2, lon2):
    R = 6371 # Earth radius, km
    lat1, lon1, lat2, lon2 = map(np.radians, [lat1, lon1, lat2, lon2])
    dlat, dlon = lat2 - lat1, lon2 - lon1
    a = np.sin(dlat/2)**2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon/2)**2
    return 2 * R * np.arcsin(np.sqrt(a))
```

Resulting `distance_km` statistics (40,344 rows):

Statistic	Value
Mean	9.72 km
Median	9.19 km
Min	1.47 km
Max	20.97 km
Std. dev.	5.60 km

All values fall within a plausible urban delivery range.

Correlation with target: 0.321. This was a *moderate* relationship, weaker than initially anticipated. Distance was expected to be the single dominant predictor; in fact it ranks below `multiple_deliveries` and `Delivery_person_Ratings` in correlation, and below traffic, festival, and city in raw effect size. The intuition holds on inspection: across a 14× increase in distance (1.5 km → 21 km) predicted time rises only about 12 minutes, and vertical scatter at any fixed distance spans 10–50 minutes. In dense Indian urban delivery, **congestion and order bundling dominate raw kilometres.**

A scatter plot also revealed distinct vertical banding at roughly 1.5, 3, 5, 7, 10, 12, 14, 17, and 20 km — consistent with coordinates generated by applying fixed offsets to base restaurant locations. Further evidence of synthetic construction; harmless for modelling.

5.2 Temporal Features

Parsing `Time_Orderd` and `Time_Order_picked` with a standard `%H:%M` format failed for thousands of rows. Diagnosis revealed **two independent defects**:

1. **Fractional-day encodings** — values such as `0.75`, `0.958333333`, `0.458333333`. These are spreadsheet-internal time representations (fraction of a 24-hour day). $0.75 \times 24 = 18:00$.
2. **Hour rollover past midnight** — values such as `24:05:00`, `24:10:00`. Valid hours cap at 23, so hour 24 denotes `00:05` on the following day.

A single parser was written to handle both, preserving day offsets so that midnight-crossing orders compute correctly:

```
def parse_time_flexible(t):
    if pd.isna(t):
        return pd.NaT
    t = str(t).strip()
    try: # fractional day
        total_minutes = round(float(t) * 24 * 60)
        day_offset, rem = divmod(total_minutes, 1440)
        hours, minutes = divmod(rem, 60)
        return pd.Timestamp('2000-01-01') + pd.Timedelta(
            days=day_offset, hours=hours, minutes=minutes)
```

```

        days=int(day_offset), hours=int(hours), minutes=int(minutes))
    except ValueError:
        pass
    parts = t.split(':') # HH:MM with possible hour >= 24
    try:
        hour = int(parts[0])
        minute = int(parts[1]) if len(parts) > 1 else 0
        day_offset, hour = divmod(hour, 24)
        return pd.Timestamp('2000-01-01') + pd.Timedelta(
            days=int(day_offset), hours=int(hour), minutes=int(minute))
    except (ValueError, IndexError):
        return pd.NaT

```

This resolved all parse failures. Zero negative preparation times resulted, confirming the day-offset handling was correct.

Three features were then derived:

Feature	Definition	Outcome
prep_time_min	Time_Order_picked - Time_Orderd	Dropped — see below
order_hour	Hour component of Time_Orderd	Retained
day_of_week / is_weekend	Derived from Order_Date	Dropped — see §6

prep_time_min proved to be pure noise. It takes only three values — 5, 10, or 15 minutes — distributed almost perfectly evenly (13,567 / 13,384 / 13,393). Its correlation with the target is **-0.01**, and group means are effectively identical (26.39 / 26.37 / 26.17). This is synthetic filler, not a measurement.

order_hour proved to be a strong feature, tracing a legible daily rhythm:

Time block	Mean delivery time	Interpretation
08:00-10:00	~19.5 min	Morning lull — fastest
11:00-14:00	~26-27.5 min	Lunch rush
15:00-16:00	~23.1 min	Afternoon lull
17:00-21:00	~27-31.2 min	Dinner rush — peak at 20:00-21:00
22:00-00:00	~22-24 min	Late night, roads clear

An 11.7-minute spread, comparable to `Road_traffic_density` — and mechanistically sensible, since dinner-rush hours concentrate simultaneous orders, traffic, and fleet strain.

6. Feature Selection

Every retention and removal decision was made against evidence — a correlation coefficient, a group-mean spread, or a distributional check — rather than intuition.

Removed

Feature	Evidence	Reason
Type_of_order	0.22 min spread across 4 categories	No measurable relationship with target
prep_time_min	$r = -0.01$; 3 discrete values	Synthetic filler
is_weekend	26.35 vs 26.21 (0.14 min)	No signal
day_of_week	2.45 min spread; no coherent pattern	Near-noise; adds 6 one-hot columns
ID, Delivery_person_ID	High-cardinality identifiers	Not generalisable features
Raw lat/long (×4)	Deterministic inputs to <code>distance_km</code>	Redundant; risk of location memorisation
Order_Date, raw time columns	Superseded by <code>order_hour</code>	Intermediate artefacts

On the raw coordinates: retaining them alongside `distance_km` is redundant by construction, and actively harmful. For linear regression, a latitude of 18.5 bears no meaningful linear relation to delivery time. For tree models, coordinates invite splits on specific restaurant locations — memorisation rather than generalisation. `distance_km` and `City` capture the geospatial signal in compact form.

Retained

`distance_km`, `order_hour`, `multiple_deliveries`, `Delivery_person_Ratings`, `Delivery_person_Age`, `Road_traffic_density`, `Festival`, `City`, `Weather_conditions`, `Vehicle_condition`, `Type_of_vehicle`

Correlation and Multicollinearity

Pearson correlation with the target, by magnitude:

Feature	r
<code>multiple_deliveries</code>	0.387
<code>Delivery_person_Ratings</code>	-0.361
<code>distance_km</code>	0.321
<code>Delivery_person_Age</code>	0.302
<code>order_hour</code>	0.185

The strongest inter-predictor correlation was `distance_km` × `order_hour` at 0.577 — moderate, plausibly reflecting longer-distance orders skewing toward evening hours. All other pairs fell below 0.13.

A VIF diagnostic error worth documenting. An initial variance-inflation-factor calculation returned alarming values:

Feature	VIF (incorrect)	VIF (corrected)
<code>Delivery_person_Ratings</code>	31.64	1.04
<code>Delivery_person_Age</code>	23.84	1.03
<code>order_hour</code>	20.39	1.50
<code>distance_km</code>	6.12	1.53
<code>multiple_deliveries</code>	2.67	1.04

These contradicted the correlation matrix, which showed no strong inter-predictor relationships. The cause:

`statsmodels.variance_inflation_factor` **assumes the design matrix already contains an intercept column.** Without one, VIF is computed about zero rather than about the mean, inflating the statistic for any variable not centred near zero — precisely the case for age (20–39), ratings (2.5–5.0), and hour (0–23).

Adding a constant via `sm.add_constant()` returned all VIFs to the 1.03–1.53 range. **No multicollinearity is present.**

7. Encoding and Data Splitting

7.1 Encoding Strategy

Feature	Encoding	Justification
<code>Festival</code>	Binary map (Yes→1, No→0)	Two classes; one-hot is wasteful
<code>Vehicle_condition</code>	One-hot	Non-monotonic (\$4.3) — must not be treated as ordinal
<code>Road_traffic_density</code>	One-hot	Medium ≈ High breaks the ordinal staircase
<code>Weather_conditions</code>	One-hot	Nominal
<code>Type_of_vehicle</code>	One-hot	Nominal
<code>City</code>	One-hot	Nominal
<code>multiple_deliveries</code>	None (kept numeric)	Genuinely monotonic: 22.85 → 47.82
<code>Delivery person Age</code> , <code>Delivery person Ratings</code> , <code>distance_km</code> , <code>order_hour</code>	None	Already numeric

`drop_first=True` was used throughout to avoid the dummy-variable trap — a redundant column producing perfect multicollinearity. This matters for linear regression and is harmless for trees.

The two encoding decisions driven by EDA — one-hot for `Vehicle_condition` and `Road_traffic_density` rather than ordinal — are

direct consequences of the group-mean analysis in §4.3, not defaults.

7.2 Splitting

A 70 / 15 / 15 train / validation / test split was used, **stratified on city** to protect the 156-row Semi-Urban category from partition imbalance.

Split	Rows	Features
Train	28,240	20
Validation	6,052	20
Test	6,052	20

Verification of stratification — Semi-Urban proportion by split:

Train	Validation	Test
0.003506	0.003470	0.003470

Near-identical, as intended.

The test set was held out entirely until §10. No model selection, tuning, or feature decision consulted it. This discipline is what makes the final number an honest estimate rather than an optimistic one.

8. Model Training

Three model families were trained in increasing order of capacity, with all evaluation performed on the validation set.

8.1 Baseline: Linear Regression

Split	RMSE	R ²
Train	5.843	0.611
Validation	5.856	0.610

The near-zero train/validation gap (0.001) indicates **no overfitting whatsoever** — expected, since linear regression has limited capacity to memorise.

But $R^2 = 0.61$ falls well short of the 0.76 benchmark, and the residual plot explains why. A healthy residual plot resembles a structureless horizontal cloud. This one showed a pronounced **diagonal band**: large positive residuals concentrated at low predictions, large negative residuals at high predictions, forming a parallelogram rather than a cloud.

That signature means the model is **compressing the true spread** — systematically underpredicting slow deliveries and overpredicting fast ones, pulling everything toward the mean. It is exactly the failure predicted in §4.1: a single linear surface cannot represent the traffic-driven bimodal regimes, the ratings plateau, or the non-monotonic vehicle-condition effect.

The baseline served its purpose: it established an interpretable floor and confirmed, empirically, the structural diagnosis made during EDA.

8.2 Random Forest

Four configurations were tested:

Configuration	Train R ²	Val R ²	Gap
n=200, max_depth=15	0.914	0.831	0.083
n=200, max_depth=12, min_samples_leaf=5	0.847	0.815	0.032
n=300, max_depth=None	0.976	0.822	0.154

Regularisation was tested and rejected. The depth-12 / leaf-5 variant narrowed the train/validation gap from 0.083 to 0.032 — but validation R^2 *fell* from 0.831 to 0.815. It traded 1.6 points of real predictive performance for a tidier-looking gap.

Methodological lesson: a train/validation gap is not inherently a defect. It is only a defect when accompanied by *worse*

validation performance. Here the deeper trees were learning genuine structure, and Random Forest's bagging already controls variance adequately at 28,240 rows × 20 features.

Pushing in the opposite direction confirmed the boundary: unlimited depth reached train R^2 0.976 with validation R^2 0.822 — a 0.154 gap and *worse* generalisation. Depth 15 sits near the optimum.

8.3 Gradient Boosting

Both models used `n_estimators=500, learning_rate=0.05, max_depth=8, subsample=0.8, colsample_bytree=0.8`.

Model	Train R^2	Val R^2	Gap
XGBoost	0.922	0.824	0.098
LightGBM	0.852	0.828	0.024

8.4 Feature Importance

Random Forest importances (top 15):

Rank	Feature	Importance
1	Delivery_person_Ratings	0.2247
2	multiple_deliveries	0.1324
3	distance_km	0.1221
4	Road_traffic_density_Low	0.1143
5	Delivery_person_Age	0.0991
6	Weather_conditions_Sunny	0.0652
7	Weather_conditions_Fog	0.0399
8	Vehicle_condition_2	0.0332
9	Vehicle_condition_1	0.0314
10	Type_of_vehicle_motorcycle	0.0242
11	Weather_conditions_Stormy	0.0191
12	Weather_conditions_Sandstorms	0.0179
13	Weather_conditions_Windy	0.0178
14	Road_traffic_density_Medium	0.0158
15	Festival	0.0140

Three observations:

Delivery_person_Ratings dominates at 0.225 — nearly double the next feature, consistent with its -0.361 correlation. The tree can exploit the plateau shape that linear regression could not.

Delivery_person_Age ranks fifth at 0.099, definitively refuting the early hypothesis that its uniform distribution implied uselessness.

Festival ranks near the bottom at 0.014 — apparently contradicting its 19.6-minute effect from \$4.3. Both facts are true, and the reconciliation matters:

Tree feature importance measures **total impurity reduction summed across all splits**. A feature that applies to only 2% of rows cannot contribute much *total* reduction, however dramatic its *local* effect. Low importance here means “rarely applicable,” not “unimportant.” On those 857 festival deliveries, **Festival** is likely carrying the prediction almost single-handedly.

The same logic explains why **City_Semi-Urban** — despite a 49.74-minute mean — does not enter the top 15 on 156 rows. **Neither feature was dropped**.

9. Model Selection

Consolidated Validation Results

Model	Train R ²	Val R ²	Gap
Linear Regression	0.611	0.610	0.001
Random Forest (depth 15)	0.914	0.831	0.083
Random Forest (depth 12, leaf 5)	0.847	0.815	0.032
Random Forest (unlimited)	0.976	0.822	0.154
XGBoost	0.922	0.824	0.098
LightGBM	0.852	0.828	0.024

Every tree model landed in a narrow **0.815–0.831** band, all comfortably above the 0.76 benchmark. This convergence signals that ~0.83 approximates the practical ceiling given the dataset’s information content — further tuning would yield fractions of a point, not leaps.

Cross-Validation

The top two candidates were separated by 0.003 on a single validation split — plausibly noise. Five-fold cross-validation on the combined train+validation set (34,292 rows) tested whether the difference was real:

Model	CV R ²	Std. dev.
Random Forest	0.8303	±0.0014
LightGBM	0.8279	±0.0010

The 0.0024 gap **exceeds both standard deviations**, indicating Random Forest held a small but consistent edge across folds — not an artefact of one split. (A prior expectation that the difference would dissolve under cross-validation was not borne out.)

Note on runtime: passing `n_jobs=-1` to both `cross_val_score` and the estimators creates nested parallelism, in which outer worker processes contend for the same cores. Setting the estimator to `n_jobs=1` and parallelising only the outer loop resolves the slowdown.

Both models were therefore carried forward to final evaluation.

10. Final Evaluation

Both models were retrained on the full train + validation set (34,292 rows) and evaluated **once** on the untouched 6,052-row test set.

Model	Test R ²	Test RMSE	Test MAE	Model file size
Random Forest	0.8302	3.864	3.108 min	149.11 MB
LightGBM	0.8302	3.864	3.122 min	1.45 MB

Interpretation

Accuracy is a dead tie. R² is identical to four decimal places; RMSE identical to three. The cross-validated edge Random Forest held did not transfer to this particular holdout split — confirming that a 0.0024 CV advantage, while statistically consistent across folds, carries no practical guarantee on any single test set. MAE differs by 0.014 minutes, under one second.

Model size is not close. Random Forest’s serialised artefact is **103× larger** than LightGBM’s. This has concrete deployment consequences:

- GitHub imposes a hard **100 MB** limit on files pushed without Git LFS. The 149 MB Random Forest artefact **cannot** be pushed to a standard repository — the usual distribution path for Streamlit, Render, and Railway deployments.
- Free-tier hosting platforms cap total slug or deployment size, often in the low hundreds of megabytes. A 149 MB model consumes most of that budget before application code and dependencies are added.
- Cold-start latency: a 1.45 MB file loads near-instantly; 149 MB adds measurable seconds to every container restart, which free tiers trigger frequently after idle periods.

Selected model for deployment: LightGBM. With tied accuracy and a 100× size advantage, this is not a close decision. It also aligns with the benchmark paper’s own conclusion that LightGBM is optimal for this dataset. The Random Forest artefact is

retained locally as a comparison reference.

What MAE = 3.11 Minutes Means

MAE is the mean absolute error: across 6,052 test predictions, the average discrepancy between predicted and actual delivery time is about 3.1 minutes. If the true time is 25 minutes, a typical prediction lands somewhere near 22 to 28 minutes.

“Average” is the operative word. Individual errors vary — many predictions land within 1–2 minutes, some miss by 5–8, and genuinely anomalous orders (extreme weather, incidents) will miss by more.

For context, commercial platforms typically display 10-minute windows (“25–35 minutes”). A point estimate carrying ~3.1 minutes of typical error is roughly three times tighter than that convention.

11. Benchmark Comparison

The reference point is *Food Delivery Time Prediction in Indian Cities Using Machine Learning Models* (arXiv:2503.15177), which evaluated Linear Regression, Decision Trees, Bagging, Random Forest, XGBoost, and LightGBM on ~45,000 records of the same Zomato delivery data, identifying LightGBM as optimal.

Metric	Published benchmark	This project	Difference
R ²	0.76	0.8302	+0.070
MSE	20.59	14.93	−5.66
RMSE	≈ 4.54 min	3.864 min	−0.67 min (≈15% lower)

Plausible Sources of Improvement

Geospatial feature engineering. `distance_km` was derived from raw coordinates via the haversine formula and ranked third in feature importance (0.122). A pipeline that feeds raw latitude/longitude directly to a model, or omits distance entirely, forgoes this signal.

Coordinate defect repair. The sign-flipped latitudes (156 rows) and near-zero placeholders (3,509 rows) were identified and handled explicitly. A pipeline that computes distance without this repair produces systematically corrupted values for roughly 8% of rows.

Temporal parsing. The dual-defect parser (fractional-day encodings plus 24-hour rollover) recovered `order_hour`, which traces a coherent lunch/dinner rush curve with an 11.7-minute spread. Naive `%H:%M` parsing silently discards thousands of rows to `NaT`.

Evidence-based feature pruning. `Type_of_order`, `prep_time_min`, `day_of_week`, and `is_weekend` were removed only after measurement demonstrated their spreads were at or near zero.

Encoding decisions grounded in EDA. One-hot encoding of `Vehicle_condition` and `Road_traffic_density` — rather than the intuitive ordinal treatment — follows directly from the observed non-monotonicity.

Caveats on the Comparison

Intellectual honesty requires stating these plainly:

This is not a perfectly controlled comparison. The paper’s exact train/test partition, random seed, cleaning choices, and dataset version are not known. Outperforming a published result on the same *dataset family* is not the same as outperforming it on *identical held-out data*. The comparison is informative, not definitive.

R² = 0.83 leaves ~17% of variance unexplained. An MAE of 3.11 minutes is a solid operational margin, not precision.

A ceiling is visible. Random Forest, XGBoost, and LightGBM all converged to 0.82–0.83 across widely varying hyperparameters. This suggests a limit imposed by the data’s information content — and given the dataset’s evident synthetic artefacts (three-valued preparation time, uniform age, perfectly balanced weather and order type, banded distances), part of that ceiling reflects signal that was never present to be learned.

12. Deployment Plan

Artefacts

```
import joblib
```

```

joblib.dump(lgbm_final, 'lgbm_delivery_model.pkl')      # 1.45 MB
joblib.dump(rf_final, 'rf_delivery_model.pkl')         # 149.11 MB (reference only)
joblib.dump(X_trainval.columns.tolist(), 'feature_columns.pkl') # 20 features

```

The feature-column list is **not optional**. Column *order* must match training exactly at inference time; without the saved schema, one-hot columns silently misalign and the model returns confident nonsense.

Inference Pipeline

Serving requires a preprocessing function that reproduces the training transformations precisely:

1. Validate incoming coordinates (reject or repair out-of-range values, as in §3)
2. Compute `distance_km` via haversine
3. Parse the order timestamp; extract `order_hour`
4. Map `Festival` to binary
5. One-hot encode the five categorical columns
6. **Reindex against `feature_columns.pkl`**, filling absent dummies with 0
7. Call `model.predict()`

Step 6 is where deployment bugs concentrate. A single order cannot produce all one-hot categories, so the raw encoded frame will be missing columns the model expects. Reindexing against the saved schema is what makes single-row inference correct.

Serving Options

Option	Trade-off
Streamlit	Fastest to stand up; usable web UI with minimal boilerplate; free hosting via Streamlit Community Cloud. Recommended.
FastAPI	Proper REST API; appropriate if the model must be called programmatically by other services
Flask	Comparable to FastAPI, more manual scaffolding

Given a 1.45 MB artefact and a demonstration-oriented goal, **Streamlit** is the natural fit. Hosting alternatives for API-based approaches include Render, Railway, and Hugging Face Spaces.

The model artefact must first be exported out of the Kaggle notebook environment — downloaded directly, or pushed to GitHub or Hugging Face — before it can be served.

13. Limitations and Future Work

Data Limitations

Substantial synthetic artefacts. Uniform age distribution, perfectly balanced weather and order-type classes, three-valued preparation time, and banded distance values all indicate the dataset was generated rather than collected. Real-world performance would differ.

Static historical data. No live traffic or weather feeds. The `Road_traffic_density` label is coarse and retrospective.

Under-represented categories. Semi-Urban (156 rows, 0.36%) and Festival (857 rows, 2%) both carry large effects on very thin evidence. Predictions for Semi-Urban deliveries should be treated as high-uncertainty regardless of aggregate model accuracy.

8% of rows discarded due to unrecoverable coordinates. If those coordinates were not missing at random, this introduces bias.

Modelling Limitations

Cyclical encoding untested. `order_hour` is treated as linear, but 23:00 and 00:00 are adjacent in reality and 23 apart numerically. Sine/cosine encoding could help a linear model; tree models are indifferent.

No `Delivery_person_ID` aggregate features. Historical per-driver mean delivery time could add signal, though it risks target leakage if computed carelessly across splits.

Interaction terms unexplored. Whether Jam traffic *during* a Festival compounds multiplicatively was not tested (EDA step 9 was skipped as unlikely to alter modelling decisions).

Point estimates only. No prediction intervals. Quantile regression or conformal prediction would let the service display an honest

range rather than a single number.

Future Directions

- Integrate live traffic and weather APIs at inference time
- Model conditional variance — residual analysis suggested mild heteroscedasticity in the reference paper’s work, and error is unlikely to be uniform across festival, weather, and distance regimes
- Evaluate on genuinely collected (non-synthetic) delivery data before drawing operational conclusions
- Add prediction intervals so users see “26–32 minutes” rather than a falsely precise “29 minutes”

14. Conclusion

A regression model predicting food delivery time was built end-to-end, from dataset selection through evaluation, achieving **$R^2 = 0.8302$, RMSE = 3.864 minutes, and MAE = 3.11 minutes** on a strictly held-out test set — exceeding the published LightGBM benchmark of $R^2 = 0.76$ on this dataset family.

The performance gain traces to disciplined data work rather than model sophistication: repairing corrupted coordinates, writing a parser for two independent timestamp defects, engineering haversine distance, and pruning four features that measurement showed carried no signal. Every tree-based model converged near 0.83 regardless of hyperparameters, indicating the ceiling was set by the data, not the algorithm.

Two diagnostic errors were made and corrected along the way, and both are documented above rather than quietly omitted:

- **A VIF calculation without an intercept column** produced values above 30, falsely implying severe multicollinearity that the correlation matrix flatly contradicted. Correcting the call returned all VIFs to 1.03–1.53.
- **Age was predicted to be worthless** because its distribution was uniform. It ranked fifth in feature importance. A feature’s marginal distribution says nothing about its relationship to the target.

Several expectations formed during EDA were also confirmed empirically: the bimodal target driven by traffic density, the non-monotonic vehicle-condition effect, and the ratings plateau above 4.5 together predicted that linear regression would underfit — and it did, at $R^2 = 0.610$, with a residual plot showing precisely the diagonal compression signature such a failure produces.

LightGBM is selected for deployment. It matches Random Forest’s accuracy exactly while producing a model artefact 103× smaller, which is the difference between fitting comfortably within free-tier hosting constraints and exceeding GitHub’s file-size limit outright.

Dataset: [saurabhbadole/zomato-delivery-operations-analytics-dataset](#) (Kaggle). *Benchmark reference:* Food Delivery Time Prediction in Indian Cities Using Machine Learning Models, arXiv:2503.15177.